

3 Predicting Trains Delays

3.1 Introduction

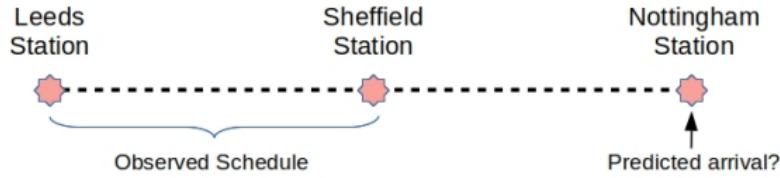


Figure 3.1.1: Diagram of Train Route

Transport systems are critical pieces of infrastructure and they have substantially increased in size in many countries worldwide. This includes rail transport systems that have evolved significantly, including to provide long-distance travel services. Train delays pose a problem to modern transport systems and rail networks, as even minor delays can cause knock-on delays throughout the system [7]. Some are technical faults or weather effects; others are failures to operate efficiently or coordinate the flow of trains [13]. Delays do not affect only the late trains; they can have a domino effect, disturbing not only the individual movements of trains on the system but the entire rail network, causing operational inefficiency and unhappy passengers [13] [7].

One of the biggest challenges for rail networks is service disruptions, which significantly impact punctuality and customer experience. These disruptions can be caused by a variety of factors, including accidents, operational issues, equipment failures, maintenance work, construction, passenger boarding delays, and extreme weather conditions. Train delays not only affect a single journey but can also create a ripple effect, impacting connecting trains and passengers' schedules. In extreme cases, prolonged delays may lead to service cancellations [19].

Given these challenges, accurately predicting train delays is crucial. Relying solely on historical average delays is insufficient, as train punctuality is influenced by dynamic factors such as passenger volume, accumulated delays from earlier trains, and weather

conditions. By developing effective delay prediction models, train operators can optimize scheduling, improve service reliability, and provide passengers with timely updates to better plan their journeys [19].

Developing great data-based models provides better predictions, which leads to an efficient railway system [13]. The traditional statistical methods used to predict delays cannot often explain the non-linear and complex relationships in the data [8] [7]. Based on that, Statistical Machine Learning (SML) techniques (like XGBoost) are more robust solutions for delay prediction that can use vast amounts of historical and real-time data.

The challenge we are dealing with deals with trains that travel from Leeds (source) to Nottingham (destination) while stopping at 4 intermediate stations. So, the journey of any train in this challenge looks like this: Leeds $\rightarrow$ Wakefield $\rightarrow$ Barnsley $\rightarrow$ Meadowhall $\rightarrow$ Sheffield $\rightarrow$ Nottingham. Our goal is to predict how early or late a train will arrive at Nottingham.

This section **3** will explain the problem statement we have dealt with and the challenges of modelling delays of trains. Moreover, we will discuss the major challenges with a challenge like this and explore different machine learning models along with the approach we took to tackle this challenge. These models have used a combination of historical data and real-time input to greatly enhance the accuracy of the delay predictions.

3.2 Problem Statement

This section aims to explain the problem we have dealt with along with some difficulties, which we think come naturally due to the nature of the challenge.

$$\frac{1}{n_{test}} \sum_{i=1}^{n_{test}} (y_i - \hat{y}_i)^2.$$

Figure 3.2.1: Performance Metrics for Trains

Figure 3.2.1 shows the performance metrics for this challenge. The performance metric calculates the mean of the sum of the squared residuals. In simple words, how much the prediction is deviating from the actual value for any particular observation. Unlike the ECG challenge, our goal here is to minimise this metric.

Train delays are a frequent problem for modern transit networks, especially for passenger and freight rail [7]. Not only do delays affect the travel experience for passengers themselves, but they also result in a decreased overall efficiency of the rail network [13] [7]. Delays of trains are caused by technical failures, extreme weather and congestion, among other reasons, making it difficult to predict when a train may not arrive on time due to their complex interactions [13] [7].

```
> head(matrixData)
```

	week.day	hour	depDelayLeeds	depDelayWake	depDelayBarn	depDelayMead	arrDelayWake	arrDelayBarn	arrDelayMead	arrDelayShef	arrDelayNotts
1	Tuesday	8	38	76	135	77	15	136	142	162	-90
2	Saturday	18	159	201	176	114	180	251	183	133	-12
3	Friday	17	33	-41	32	144	-78	27	79	244	171
4	Tuesday	8	18	80	138	48	6	74	112	132	-164
5	Friday	18	4	15	15	-37	8	77	28	62	-111
6	Sunday	11	-109	168	409	431	-52	466	505	483	786

Figure 3.2.2: Delays of First 6 observations

In addition, the task of predicting train delays changes drastically with geography. This is because the definition of delay varies from nation to nation. A delay of 5 hours might be very unusual in one nation while it may be completely normal in another.

Furthermore, the anomalies associated with this type of challenge are insane. First of all if we have a look at the delays of the first observation in the training dataset shown in figure 3.2.2, then we can notice that the train arrived and departed late at every station and yet it arrived early at Nottingham. This clearly shows that there are certain things that might have influenced the performance of the train between Sheffield and Nottingham, which may or may not be predictable. This is still believable and predictable, in some countries, a train will be running late by 10 hours at night, and the very next morning it would have arrived 2 hours early at the destination. This is a fact and not an exaggeration. What happens overnight is still a mystery even to passengers. Now imagine trying to predict whether a train will arrive early or late using the data collected in that

country.

3.3 Dataset Overview

This section aims to explain the data set that was given to us for this challenge.

```
> print(trainingData[[1]])
$timings
  train.code day.week departure.from departure.time departure.schedule arrival.to arrival.time arrival.schedule
1    1Y09   Tuesday      LEEDS      07:05:38      07:05:00    WKFLDKG    07:22:15    07:22:00
2    1Y09   Tuesday    WKFLDKG    07:25:16    07:24:00      BNSLY    07:38:16    07:36:00
3    1Y09   Tuesday      BNSLY    07:42:15    07:40:00    MEADWHL    07:54:22    07:52:00
4    1Y09   Tuesday    MEADWHL    07:57:17    07:56:00    SHEFFLD    08:02:42    08:00:00

$congestion
 week.day hour Leeds.trains Leeds.av.delay Sheffield.trains Sheffield.av.delay Nottingham.trains Nottingham.av.delay
1 Tuesday    8           39              0              22              0              13              0

$arrival
 arrival.to arrival.time arrival.schedule delay.secs
1      NTNG    08:56:30    08:58:00          -90
```

Figure 3.3.1: First Observation of the Training Dataset

The overall dataset is divided into three parts: training dataset (trainingData), test dataset (testData) and historical congestion. The training dataset has 1859 observations while the test dataset has 465 observations. The historical congestion is a separate variable storing historical data related to congestion and has 126 observations.

The training and test data set has two types of data, the first is endogenous data, which means factors that directly affect train delays, such as arrival and departure delay [19]. The training data includes the following information:

- Arrival and departure timings with original schedule for the train
- Arrival schedule and time at Nottingham, with its associated delay
- Congestion summaries with numbers of trains and average delay across the 3 major stations. A clear explanation on what these mean is given in section 3.6.

The test data have the same format, but are missing the arrival time and delay at Nottingham, which we need to predict. The second type of data is exogenous data, which means external factors that affect train delays, such as historical congestion, weather, distance, etc.[19] Our dataset only has historical congestion, which has summary information on historical congestion data across weekdays and hours of the day. Additional explanation

about the structure of historical congestion is given in section **3.6**. The training dataset is used to train the model while the test dataset is used to evaluate the performance of the model.

3.4 Models and Methods

This section aims to briefly explain some machine learning models and methods that we have used in case the reader is not familiar with them already.

XGBoost →

Description for this model is detailed in section **2.4**.

GAM →

Generalized Additive Models (GAMs) are an extension of traditional regression models that allow for more flexibility by replacing strict linear relationships with smooth, non-parametric functions. Unlike conventional models, which assume that the effect of each predictor on the outcome follows a fixed, linear pattern, GAMs use smooth functions to model relationships dynamically. This makes them especially useful for capturing complex, nonlinear trends in data while maintaining interpretability. The key idea is to estimate these smooth functions iteratively using scatterplot smoothing techniques, which adjust the model to fit the data more naturally [9].

Recursive Feature Elimination (RFE) →

Feature selection (FS) is a technique used to identify the most important features in a dataset while removing irrelevant or redundant ones. The goal is to select the features that contribute the most to predicting the target variable, making the model more efficient and accurate. With the rapid growth of data collection across various fields, datasets are becoming larger and more complex. This increase in data volume requires more computing power and longer processing times for machine learning models. In many cases, the challenge is not just building a predictive model but selecting the right subset of features that best represent the data while eliminating unnecessary ones [20].

3.5 Exploratory Data Analysis (EDA)

This section demonstrates some key analysis and findings over the 5 week period. This section is closely related to the section 3.6 and the reader will find some explanations and insights that will help to understand the workflow of the previously mentioned section.

Train Code Analysis →

In week 2, we were wondering if the individual train code could be put to any use in our modeling. Therefore, we decided to determine how many unique trains there are in the dataset.

```
> table(days)
days
Friday    Monday  Saturday    Sunday  Thursday    Tuesday  Wednesday
   274       289       226       186       298       281       305
```

Figure 3.5.1: Number of Trains on each Day

The figure 3.5.1 shows the number of trains that run on each day. The numbers are more or less in the same range, with Sunday and Wednesday acting as exceptions. This shows that Sunday has the least trains running due to being a weekend and Wednesday has the most trains running probably due to some reasons unknown to us.

```
> print(train_occurrences)
      Day
TrainCode Friday Monday Saturday Sunday Thursday Tuesday Wednesday
1M01         0         0         0         4         0         0         0
1Y05         17        21         0         0        24        18        23
1Y09         20        19        14         0        22        18        23
1Y13         17        25        16         0        23        20        18
1Y17         22        21        20        21        17        15        23
1Y21         21        22        18        18        21        21        19
1Y25         21        20        19        20        24        22        23
1Y29         16        18        20        17        18        20        21
1Y33         22        23        18        16        25        18        23
1Y37         23        21        17        15        20        19        23
1Y41         18        18        14        21        21        24        23
1Y45         21        19        18        19        22        21        24
1Y49         17        22        17        18        22        22        19
1Y53         19        18        18         0        23        19        21
1Y57         20        22        17        17        16        24        22
```

Figure 3.5.2: Occurrence of individual Train on each Day

The figure 3.5.3 shows the occurrences of each train running on each day. According to the figure, there are a total of 15 unique codes, indicating 15 unique trains that run throughout the week.

Now the question is how to use them in our challenge. We can possibly extract the train code and the day on which it runs from the test set, but what will we do with that information? First of all, we are not even sure if the same information about train will be true for the instances in the test set or not. For example, according to the figure 3.5.3, our dataset has instances of the train "1M01" running only on Sunday, but does this mean this train does not run on any other day or the test set will not have any other instance of this train running on any other day.

This implies that we cannot predict the delay of an instance based on the train code it has, since the information about train code can vary and is not reliable. What we mean is that if trains are generally late on Sunday, then it is going to be the same for every train, and the individual train code has nothing to do with it.

Day Encoding Analysis →

In week 2, we wanted to encode the day variable in a way that would help the model make better predictions. We therefore plotted the following plot which shows the distribution of delay on each day.

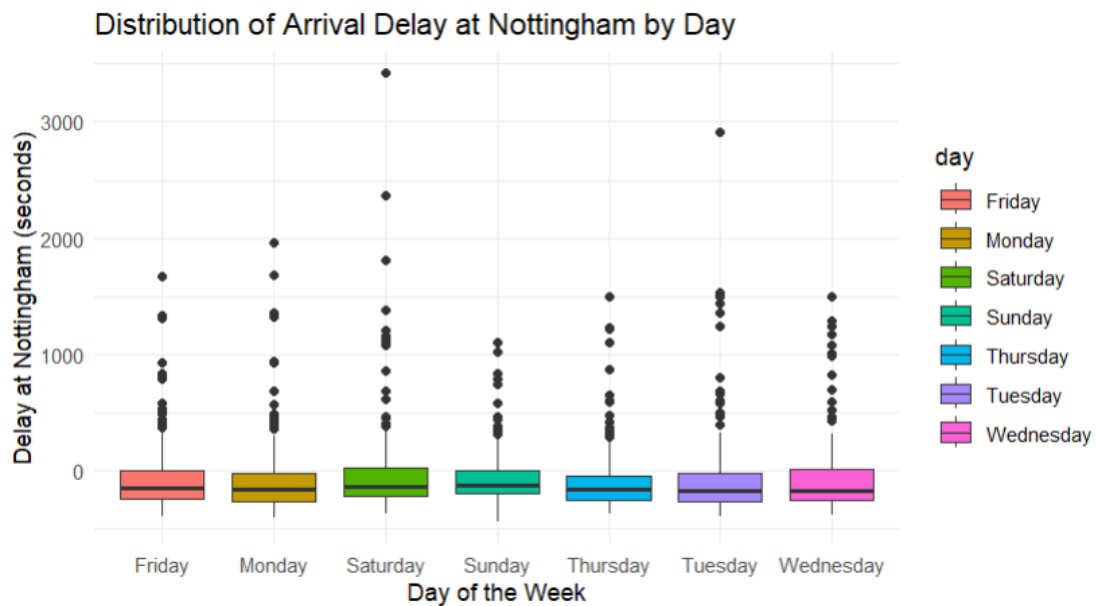


Figure 3.5.3: Distribution of Delay for each Day

Upon close observation, it can be noticed that the median value of weekends seem to be a bit higher than those of the weekdays. Moreover, the weekday boxes are very similar, indicating that they can be combined together. On the other hand, the weekend boxes almost have the same median value, however, the sizes of the boxes are different. This means that we can either combine the weekends to make a single category or we can also try to keep them as two separate categories.

This analysis led to three approaches, and to select the best we relied on correlation value. The three approaches along with the chosen approach and the correlation value are mentioned in section 3.6.

Outlier Removal →

In week 3, we thought about applying outlier removal to make better predictions. One of our teammates learnt about two common outlier removal techniques in the Machine Learning module during the autumn term. These two techniques are the Z-Score and IQR method. We don't have in depth knowledge or understanding of how these methods work, however, we just have basic understanding of these two methods and when to apply them.

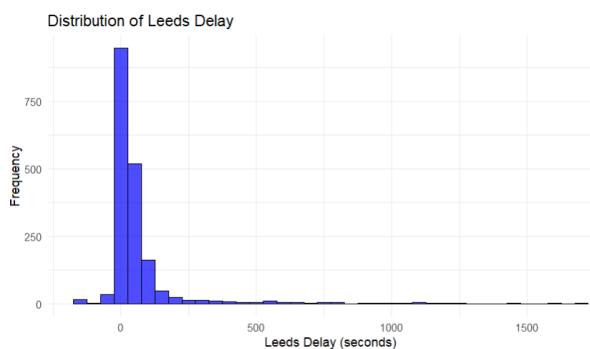


Figure 3.5.4: Leeds Departure Delay

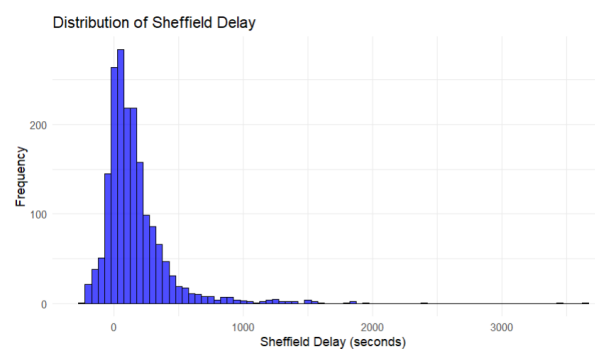


Figure 3.5.5: Sheffield Arrival Delay

Z-Score is generally applied when the distribution is normal, while IQR is applied when the data is skewed. From the figures 3.5.4 and 3.5.5, it appears that the distribution is right-skewed. This remains true for the other 8 features related to delay, however, the same is not true for the 8 features extracted using congestion and historical congestion.

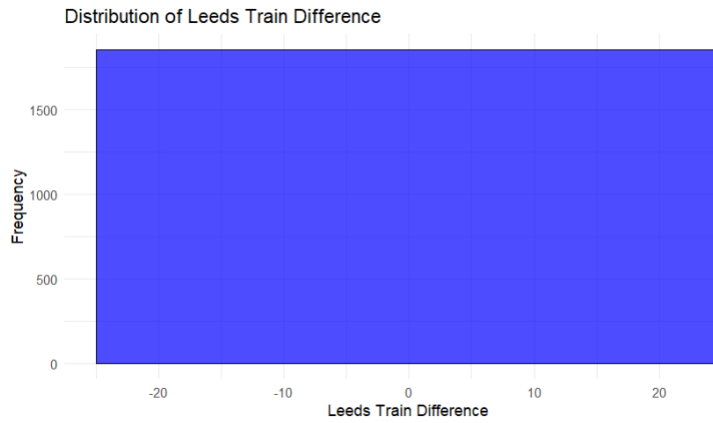


Figure 3.5.6: Leeds Train Difference

The distributions of these 8 features were strange, suggesting a constant distribution. One of these is shown in figure 3.5.6. We were not sure what to make of this and therefore, decided to apply the same outlier removal technique on these features as the other 8 features related to delay.

In the end, the IQR method was applied to all 16 numerical features to remove outliers due to skewness in the distribution of the features. At this stage we were done with feature selection and did not have any categorical feature as we dropped them already, and therefore, we didn't need to perform any analysis to find any rare values in those categorical features. Feature selection is mentioned next to ensure consistency with the analysis of PCA.

Feature Selection →

In week 3, we decided to perform feature selection as we believed we had already extracted all possible features from the training dataset. We divided the analysis into two parts, the first for numerical features and the second part for categorical features.

Numerical Features

At this stage, we had 16 numerical features in the train set. To analyse the predictive power of the numerical features, we relied on the correlation value with the target variable. In addition, we plotted the scatter plot to have a deeper insight of the distribution

of the data.

```
+ }  
Correlation of depDelayLeeds : 0.3752154  
Correlation of depDelayWake : 0.5267219  
Correlation of depDelayBarn : 0.619896  
Correlation of depDelayMead : 0.7415907  
Correlation of arrDelayWake : 0.5139324  
Correlation of arrDelayBarn : 0.6207663  
Correlation of arrDelayMead : 0.7306231  
Correlation of arrDelayShef : 0.8124464  
Correlation of hour_sin : -0.09454259  
Correlation of hour_cos : 0.08764204  
Correlation of Leeds_train_diff : 0.001009545  
Correlation of Sheff_train_diff : -0.1270098  
Correlation of Notts_train_diff : -0.05507779  
Correlation of Leeds_delay_diff : 0.2602094  
Correlation of Sheff_delay_diff : 0.2387868  
Correlation of Notts_delay_diff : 0.05897982  
Correlation of high_congestion : -0.00438197  
Correlation of high_delay : 0.1835462  
Correlation of day_encoded : 0.06570034  
> |
```

Figure 3.5.7: Correlation with Target Variable

After analysing the figure 3.5.7, it is clear that there are only 7 features which can be attributed as strong features if we set a correlation threshold of 0.5 and -0.5. Upon this conclusion, we retained only 7 features and observed the performance of the model and there was a considerable increase in the MSE. This implied that those features were actually contributing positively to the performance of the mode. Moreover, this made us realise that correlation alone does not reflect the predictive power of a feature.

The correlation explains the change in the target variable with the change that occurs in the feature. This means how the value of the target variable is changing with a decrease or increase in the value of the feature. So, if the correlation value is closer to 0, then it means that there is no change in the value of the target variable with the increase or decrease in the value of the feature, however, this experience taught us that this does not mean that the feature with correlation value of 0 is bad. This just means that there is no significant pattern in the change that occurs.

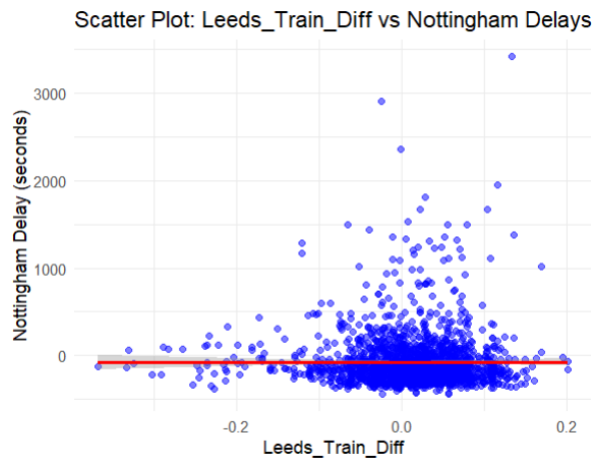


Figure 3.5.8: Leeds Train Diff and Nottingham Delay

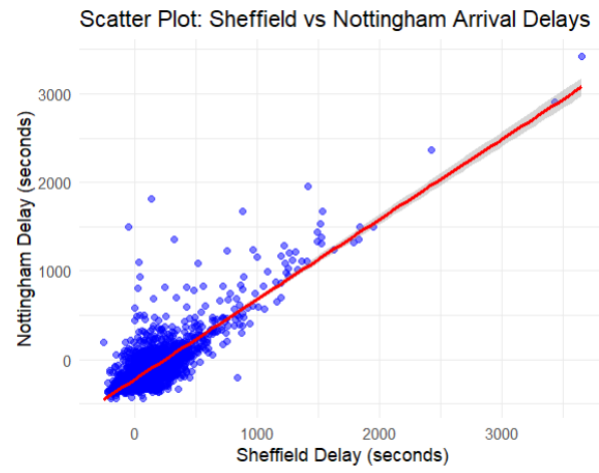


Figure 3.5.9: Sheffield Arrival Delay and Nottingham Delay

Figures 3.5.8 and 3.5.9 show two distinct features where one has a negligible correlation value of 0 and the other has a high correlation value of 0.81. The regression line shows the change that occurs in the target variable with the increase in the value of the feature. In the first figure, there is no change occurring in the target variable, whereas in the second figure the value of the target variable is increasing. From the point of view of correlation, the former feature is a bad feature and the latter is a good feature.

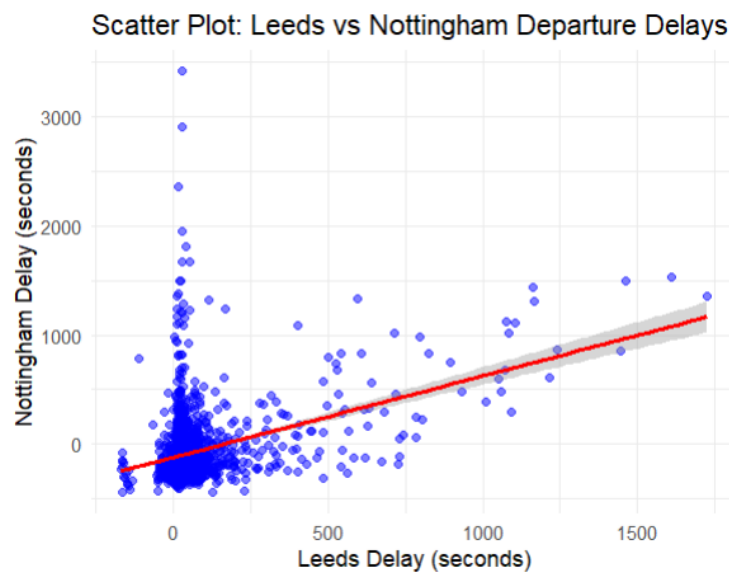


Figure 3.5.10: Leeds Departure Delay and Nottingham Delay

Although, according to correlation, Leeds departure delay is a less powerful feature due to its poor correlation value of just 0.37, it still looks like the feature is decent enough to add some value to the model's performance. This conclusion was drawn after having a look at the scatter plot shown in figure 3.5.10. This implies that a correlation value of 0.3 is not as small as one might think.

In addition to all of this, we noticed one more thing that the feature that was used in week 1 with the Linear Regression model has a linear relationship with the target variable, which is shown in figure 3.5.9. This explains how a basic model like Linear Regression achieved a decent MSE of 30700 on the test set using only one feature.

Categorical Features

At this stage, we had 3 categorical features in the train set. To analyse the predictive power of categorical features, we used the box plot.

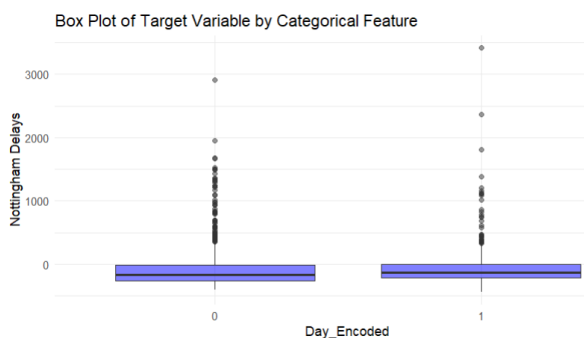


Figure 3.5.11: Box plot of Day Encoded

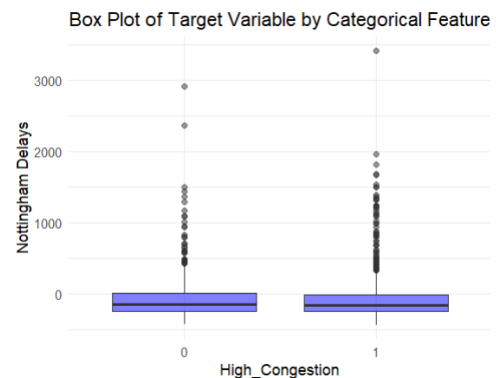


Figure 3.5.12: Box plot of High Congestion

Figures 3.5.11 and 3.5.12 illustrate the box plot of two categorical features. It seems that there is no difference or negligible difference in the median values of 0 and 1 in both plots. Moreover, there is a complete overlap between boxes 0 and 1 in both plots. This suggests that the model cannot differentiate between 0 and 1 for both features, which implies poor predictive power.

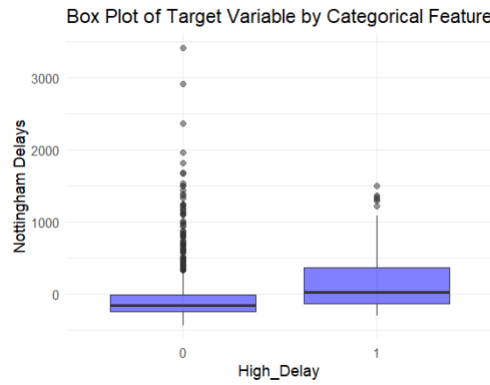


Figure 3.5.13: Box plot of High Delay

The plot shown in figure 3.5.13 shows that there is a considerable difference between the median values of 0 and 1, however, there is still some overlap between boxes 0 and 1, indicating that this feature is not as poor as the previous two, yet it is not a strong standalone feature.

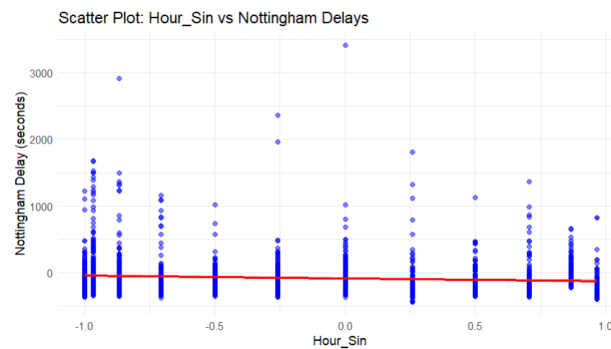


Figure 3.5.14: Scatter plot of Hour Sin and Nottingham Delay

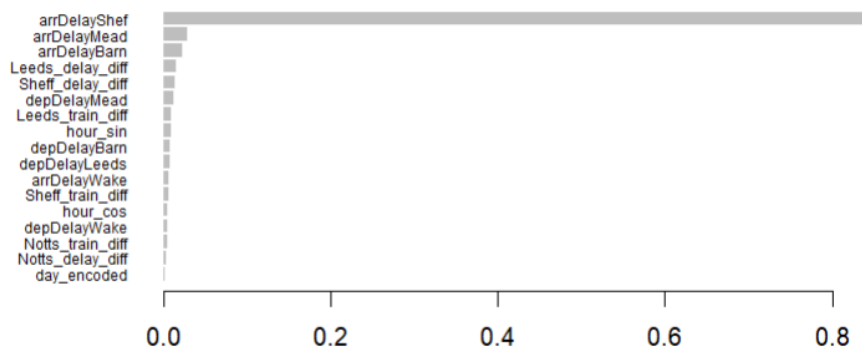


Figure 3.5.15: Feature Importance Plot from XGBoost

The box plots were not very encouraging and the scatter plot of certain numerical features were difficult to comprehend, as shown in figure 3.5.14. Performing feature selection solely on the correlation value resulted in a performance degradation. This led us to look for a different way to perform feature selection. We then observed the feature importance plot of XGBoost and tried to extract any pattern. This plot is shown in figure 3.5.15.

From the plot, we could observe that the categorical features were contributing the least to the model's performance, while there were certain other questions that were coming to our mind. One notable among them was how only one feature contributes so much, while others are not contributing at all. We ignored this question for now and focused on choosing the features that were adding value to model's performance. The other two categorical features are not visible in the plot, so we referred to the importance matrix. After confirming this observation from the importance matrix we decided to drop the three categorical features as they were the last three features in the plot and matrix. Additionally, we also deduced that categorical features will not prove to be very beneficial in this challenge since the box plots are not strong and the feature importance plot indicates the least contribution as well.

PCA Analysis →

In week 4 and 5, we gave PCA a consideration which is a dimensionality reduction technique and, as a result, did the analysis for the same two different times. This analysis has been divided into two parts for better clarity and understanding.

Analysis in Week 4

In week 4, we considered applying PCA to the 16 numerical features that we had in our train set. PCA is typically applied in three scenarios: when the dataset is highly dimensional, which means there are a lot of features, or when the features are redundant, which means that the features are highly correlated to each other, or when there is a lot of variance in the features of the dataset.

In our case, there are only 16 features which do not qualify to be called as highly dimensional. We checked for high collinearity or redundancy and found out some features are indeed correlated to each other, however, all of those features were related to arrival and departure delay. This means technically they are correlated but should we consider them as correlated features that needs to be removed?

So, we chose not to consider them as correlated features and the reason for this is that we are dealing with "Trains Challenge" where the train moves from one station to the other. So, if a train was late at station A by 50 seconds, then the majority chances are that it will be late at station B as well and the delay will be in the zone of 40 to 60 seconds. This will technically make the features related to delay of these 2 stations correlated, however, this is the nature of the problem and hence dropping any of them would be a bad move. We can be wrong about this, but this is what we thought during the model development phase and at that time this logic made sense to us.

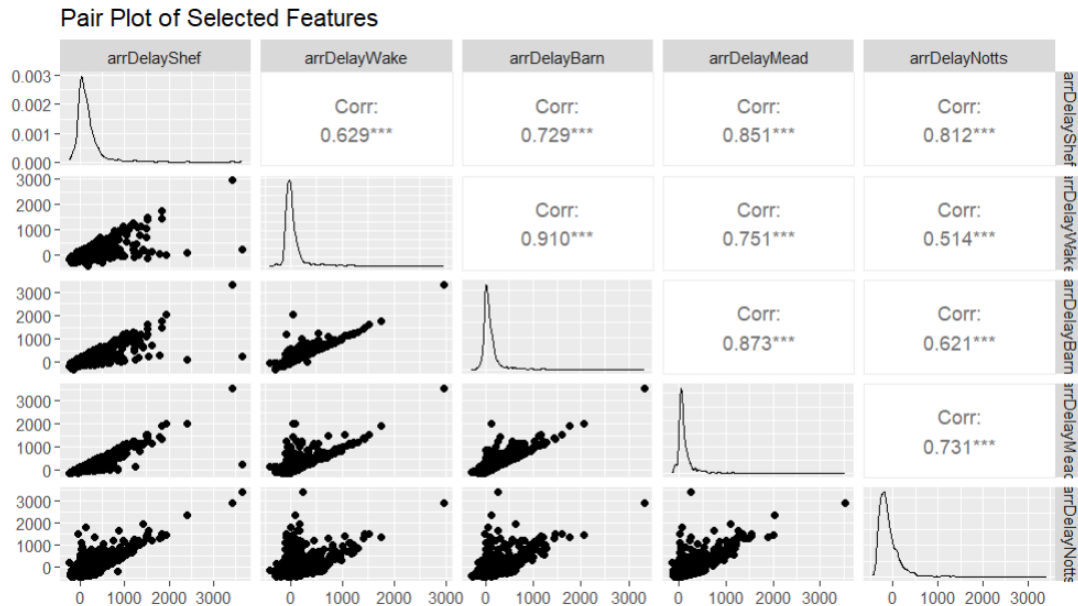


Figure 3.5.16: Pair Plot of Arrival Delay

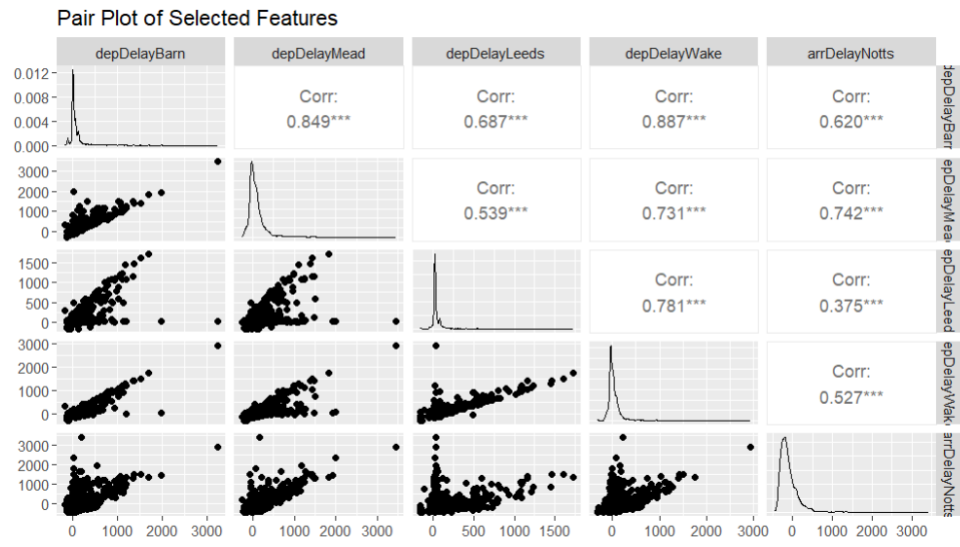


Figure 3.5.17: Pair Plot of Departure Delay

With this, 2 of the 3 conditions are not met, and the last one was checking about variance. We observed pair plots 3.5.16 and 3.5.17 as they consist of multiple scatter plots in a single plot, making it easier to visualise. From the plots it appeared to us that no matter the feature, the data points are not scattered randomly with much variance. Moreover, the data points formed a large cluster to the bottom left where the majority of the data points were located.

We needed to observe the last row of the pair plot as that is the row that shows the scatter plot between the feature and the target variable. From our understanding of variance, the plots did not show much sign of it. We mean that can we really call that variance, off course there would be some points that will be a bit far from the majority of the points or might be some features that have no significant pattern, but does this mean we can say that the dataset has features with a lot of variance. To us it did not appear that any of the features is having no significant pattern in distribution of the data points and therefore, we deemed that there is no significant variance in the features which might force us to apply PCA.

We could deliver this explanation during our presentation clearly and as a result we decided to take another approach to explain it here. To explain this, we tried applying

PCA to the 16 numerical features and tried capturing as much variance as we can using 10 principal components. The variance captured by the individual component as well as the total variance is shown in figure 3.5.18.

```
> cat("Explained Variance by Top 10 PCs:", explained_variance, "%\n")
Explained Variance by Top 10 PCs: 41.67 9.452 9.133 7.417 6.227 5.665 5.461 4.57 3.355 3.163 %
> cat("Total Variance Explained:", sum(explained_variance), "%\n")
Total Variance Explained: 96.113 %
>
```

Figure 3.5.18: PCA applied on 16 features

Now, from the figure, we can draw some conclusions. Firstly, it is clearly apparent that to capture the 96% variance, we had to use 10 principal components when our train set had only 16 features. This means that PCA is applied to reduce dimensions, so if we need 10 principal components to capture 96% variance when originally we had 16 features, then there is no point in applying PCA, as there is not much difference between 10 and 16. Secondly, The first principal component could only capture 41% variance, which means that not a lot of data are distributed in one direction, however, this also means that the features are not highly correlated.

These two conclusions and the previous analysis from the pair plot and reasoning make this clear that none of the requirements for applying PCA are met. We can be wrong about our reasoning, deductions, and analysis, but this is what struck our mind during working on the challenge, and therefore we concluded that applying PCA will not be helpful in our case.

Analysis in Week 5

In week 5, we gave PCA another consideration since we had 133 features which certainly can be termed as highly dimensional, and most of the features were created by interaction using 16 initial features, so it was obvious to have a lot of correlated or redundant features.

```

> explained_variance <- summary(matrix_pca)$importance[2, 1:20] * 100
> cat("Explained Variance by Top 20 PCs:", explained_variance, "%\n")
Explained Variance by Top 20 PCs: 26.933 13.353 7.247 5.098 4.199 3.476 3.25 3.072 2.923 2.63 2.48 2.18 2.093 1.948 1.641 1.52 1.32 1.168 1.136 1.023 %
> cat("Total Variance Explained:", sum(explained_variance), "%\n")
Total Variance Explained: 88.69 %

```

Figure 3.5.19: PCA applied on 133 features

We tried capturing as much variance as we can using 20 principal components which is shown in figure 3.5.19. The results are quite intriguing, this is because only 20 principal components can explain most of the variance in the dataset and only the 12% variance is hidden in remaining 114 features, which was expected. However, the first principal component is just capturing 26% variance, which implies two things. One, no single direction dominates the data, which means that the data is not distributed in a single direction, rather it is distributed in many directions in a complex way. Second, the features are not highly correlated, which is surprising.

In week 4, we were left with 49 features after removing redundant features, and the threshold was set to 0.8 which is quite high. This means that the features were definitely correlated. Moreover, these 117 new features were created using the initial 16 features via mathematical operations, so logically it makes sense that they will be correlated. But PCA tells a completely different story. This can be due to two reasons. First, our understanding of the first principal component is wrong. Second, there is something more about the working of PCA that is beyond our knowledge.

Whatever the case might be, we could understand that the dataset is highly dimensional due to 133 features and that the first principal component is struggling to capture a lot of variance, which implies that the data is distributed in many directions, potentially indicating large variance. On the basis of these two reasons, we decided that PCA might be a sensible move this time and it worth trying atleast once. Therefore, we tried applying PCA and the performance of the model degraded. With this we learnt that PCA is not something that can be applied just because the dataset is highly dimensional, or the features are highly correlated or maybe there is a lot of variance.

Analysis of Distribution of Target variable →

During the challenge, we were always stuck between the MSE range of 28000 and 30000. As a result, we started wondering what are we doing wrong and how we can improve our model's performance. Therefore, we plotted the distribution of the target variable.

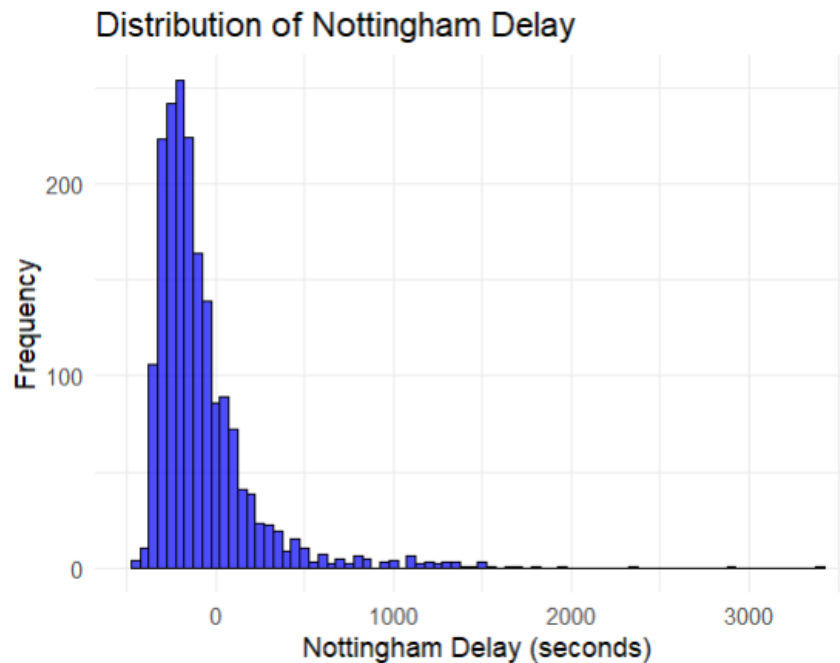


Figure 3.5.20: Distribution of Nottingham Delay

The figure 3.5.20 clearly shows that the distribution is right-skewed. But that is not all, most of the delays we have in the dataset are around 0. In week 4 we realised why the evaluation metrics for this challenge is chosen as MSE. Considering that we don't have a lot of instances for those extreme delays to the right, it is difficult for the model to learn better to make good predictions for those values. This is one of the reasons why we achieved such a bad MSE in week 3 because we removed those extreme values and the model never got the opportunity to learn them.

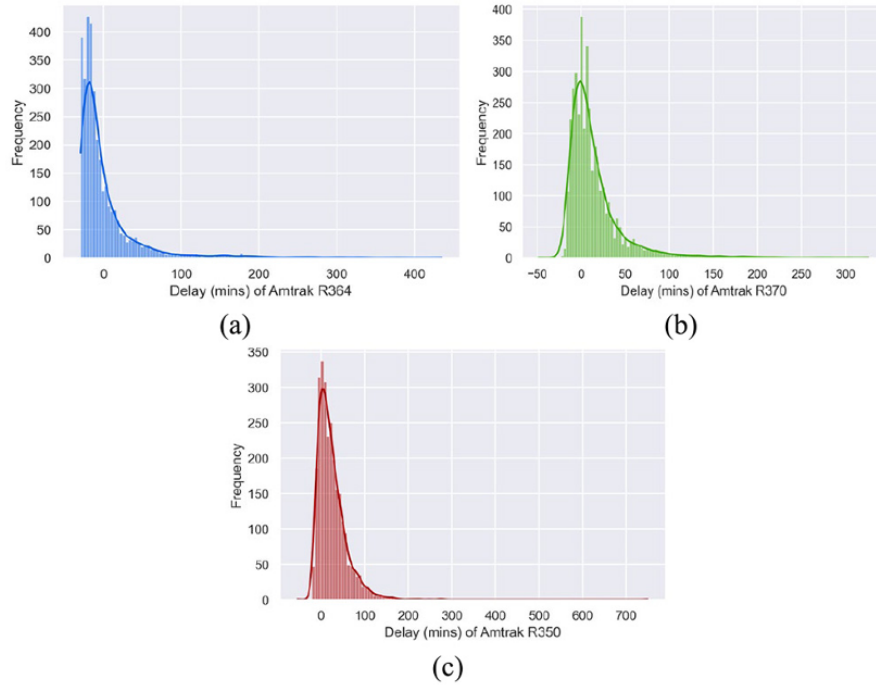


Figure 3.5.21: Distribution of Amtrak Delay [19]

Now that we identified what the biggest challenge was in this challenge, we now needed to understand how to deal with it. Therefore, we referred to some papers to see if someone has encountered a similar problem. One of the papers [19] tackled a similar problem as ours where the target variable is skewed. Unfortunately, no mention of how to deal with this skewed data was made. What is surprising is that they have the same amount of data available for those extreme values and they had an MAE score in the range of 10 while ours was above 100. We could not understand what kind of clever and intelligent modeling they did, as there was no vital information about this in the paper. One thing we learnt was why they chose Random Forest and Gradient Boosting Machine, and that is because tree-based models could handle skewness in the data easily.

Weights Analysis for Blended Approach →

In week 5, we decided to deploy the ensemble approach. The approach we chose was a very basic blended approach in which the predictions made by both models are multiplied by a weight, and then the result is added to get the final value. The plot has been shown in figure 3.5.22.

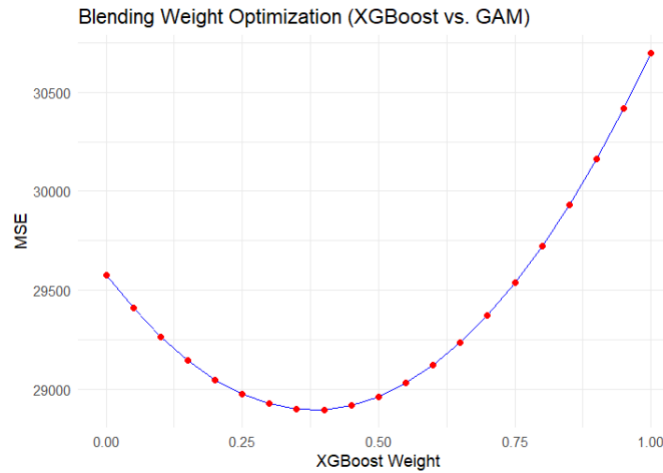


Figure 3.5.22: Performance with different Weight Combinations

According to the plot, XGBoost should be assigned a value of 0.4 to obtain the optimal result, which means that GAM should be assigned 0.6. Although this weight combination gives the best MSE on the evaluation set, there is no guarantee that the same will be true for the test set as well. Now that we have already identified that the main challenge to tackle is those extreme positive values, it makes much more sense to choose a model that can better predict those extreme values with existing data or in other words has better MSE value.

Therefore, we decided to deploy a blended model with a weight combination of 0.2 for XGBoost and 0.8 for GAM since GAM had better MSE than XGBoost. The reason we chose to keep XGBoost in the blend with minimal weight is because XGBoost had better MAE than GAM. This implies that XGBoost is making better and consistent predictions overall while GAM is making better predictions for those extreme positive values. We believed that this approach and the new weight combination would give us the best of both worlds and help us achieve the best MSE possible.

With this we concluded our EDA for this challenge. In short, we had a clear understanding of how to approach the challenge, moreover, we could understand why certain actions made sense and what might or might not work. Unlike the ECG challenge, the process here was more organised driven by logic, reasoning, and analysis.

3.6 Progress Over 5 Weeks

This section discusses the various steps we took to make our predictions each week, along with the thought process and ideas that came to mind. The workflow for each week begins with an analysis of the results and steps taken in the previous week, followed by the development of new approaches for the current week. This section is closely related to section 3.5, and the reader will need to refer back to that section frequently to gain a thorough understanding of the workflow. The previously mentioned section also contains explanations that provide additional insights.

Preliminaries →

The following are some essential terminologies and concepts that will help the reader follow the discussion effectively.

- Training Dataset → This refers to the initial dataset which obtained after loading the RData for this challenge.
- Historical Congestion → This refers to the variable which is obtained after loading the RData for this challenge and contains historical data related to congestion.
- Train Set → This refers to the set which is obtained after the 80-20 split and is used for training the model.
- Evaluation Set → This refers to the set which is obtained after the 80-20 split and is used to evaluate the performance of the model on our system.
- Test Set → This refers to the "testData" variable on which we made predictions for five weeks.
- The same preprocessing steps were applied to the test set as to the train set, unless otherwise specified.
- The last 2 digits of all MSEs obtained on the evaluation set has been rounded to zeroes, unless otherwise specified.
- The "**Result**" section in each week indicates the performance of the model on the test set.

3.6.1 Week 1

Since this was the first week, we did not have any prior results to analyze. Therefore, our focus was on planning the steps for the current week.

Following our approach from the ECG challenge, we began by establishing a clear roadmap for the upcoming weeks while ensuring a thorough understanding of the problem statement. Additionally, we made an effort to anticipate potential challenges that might arise during the project.

This challenge differed from the ECG challenge in terms of problem type. In the ECG challenge, the lack of domain knowledge prompted us to gather relevant research papers to understand the problem better. Although this challenge did not require significant domain knowledge, we still considered reviewing previous works to gain insights into how similar problems were approached.

We started by carefully reading the challenge sheet provided on Moodle, which outlined the challenge and offered suggestions to serve as a foundation for our workflow. After thoroughly examining the challenge sheet and ensuring our understanding of the task, we began by identifying potential difficulties before diving into the technical aspects of the challenge.

The first major difficulty we identified was understanding the dataset. This was our first encounter with a dataset consisting of two distinct components: the primary training dataset containing three variables, and another variable comprising additional historical data collected over previous years, referred to as historical congestion.

Since this was the initial week, we set aside the historical congestion data for later analysis and focused exclusively on the training dataset. The training dataset consisted of three variables, each with several columns. For this preliminary stage, we chose not to overthink model selection and opted to use a basic Linear Regression model to submit

our predictions. This decision allowed us to understand the complexity of the challenge.

Relying heavily on the suggestions from the challenge sheet, we employed the provided code to convert time to seconds, extract one feature, and select the target variable. Following the standard workflow of splitting the data and training the model, we achieved a Mean Squared Error (MSE) of approximately **34,000** on the evaluation set.

Some of our teammates had experience with regression problems and understood that regression is generally more challenging than classification. Since we lacked a baseline performance metric for datasets related to trains, we referred to previous research papers to observe their performance metrics on similar datasets. Surprisingly, these papers reported Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) scores ranging from 10 to 20 [19]. This discovery indicated that achieving a low error rate would be highly challenging, and our MSE of 34,000 suggested that substantial effort would be required to improve performance.

Result →

The basic Linear Regression model trained using only one feature achieved an MSE of **30,700**.

3.6.2 Week 2

We began this week by analysing the results and steps taken in week 1. We were amazed to see the MSE our model scored on the test set because the model's performance is generally worse on the test set than the evaluation set. This means we were expecting an MSE of more than 34000. Notably, there were 1859 instances in our training dataset, and after the 80-20 split, we were left with 370 instances in our evaluation set. The test set had 465 observations, meaning our model was performing better on the test set with more observations than the evaluation set with fewer observations, which is quite rare.

This week, we dedicated our complete attention to feature engineering, as we believed that

many powerful features that better capture relationships would be the key to achieving better performance. We first started by engineering features that were easy to engineer, like delays. There were 2 types of delays: departure and arrival delays. Excluding the target variable, we had 8 features related to delay: 4 for delay related to arrival and 4 for delay related to departure.

These delays were extracted from the timings variable present in the training dataset. Apart from delays, there were 3 more features: train code, day of the week, and name of the stations. The name of the stations did not seem useful, so we ignored it. Then we started wondering if the train code could be useful and how many unique codes there were. Therefore, we calculated the number of unique trains and the day on which they were running. Additionally, we calculated the number of times each train was running on every day. This calculation can be found in section **3.5**.

From the analysis, we couldn't derive any meaningful information that might prove useful, so we concluded that specific train codes were of no use. We then started wondering how individual days could be used to extract some features that might increase our model's performance. Consequently, we noticed that day is also present in the congestion variable of the training dataset, making us wonder why the same information for every instance is given twice.

We then began analysing the congestion variable in the training dataset. After examining the 6 columns present in the variable apart from day and hour, we wondered what they meant and how we could use them. Subsequently, we realised that historical congestion sounded similar to congestion and might be linked to it. The very first thing that struck our minds was, "What is the difference between congestion and historical congestion?" Therefore, we referred to the challenge sheet.

After examining the columns of historical congestion, we realised that congestion and historical congestion had the same columns. We started to understand why the day

column is present in both timing and congestion variables. We felt the need to join 2 tables using day and hour as conditions, similar to joining tables in SQL using the "where" command. After carefully examining the historical variable, we realised there were unique entries for every hour of any particular day, where the hour variable ranges from 6 to 23 for every day. This gives us 18-hour entries for 7 days, resulting in $18 * 7 = 126$ entries.

We then linked congestion and historical congestion using day and hour as conditions. For instance, if an observation in the training dataset has the week value as Tuesday and hour value as 8 in the congestion variable, then that particular instance is linked to the entry in the historical variable where both day and hour values are the same. After linking, our task was to extract meaningful features for our train set.

We observed 6 columns where there were 2 types of columns for 3 stations. The 2 types were average delay and average number of trains present at the station. We decided to extract this information as ratios. While it made sense to extract the average number of trains as a ratio, it did not make much sense to do the same with the average delay. Since we could not come up with any other way of manipulating average delay, we decided to let it be as a ratio. Consequently, we extracted 2 more categorical features using these 6 features. If the present average delay is greater than the historical average delay, then the value is set to 1, otherwise, 0. Likewise, if the present number of trains present at the station is greater than the historical value, then the value is set to 1, otherwise, 0.

With this, we extracted 8 new features using congestion and historical congestion. One of the papers [19] suggested using hour as sin and cos transformations because 6 is further away from 23 than 11 on the number line. However, 6 is closer to 23 than 11 on the clock due to the cyclic nature of the clock. Additionally, the paper [19] also recommended encoding day as a categorical variable, however, there was no specific information on how to encode it.

Therefore, we tried encoding the day column in 3 different ways. Firstly, all days re-

ceive a unique code from 0 to 6. Secondly, Saturday is encoded as 0, Sunday is encoded as 1, and weekdays are encoded as 2. Thirdly, the weekend is encoded as 1 while weekdays are encoded as 0. To verify the effectiveness, we calculated the correlation of each approach with the target variable and discovered that the third approach had the highest correlation of **0.06**. Therefore, we chose to proceed with that encoding approach for the day column. These approaches were framed based on an analysis that is shown in section **3.5**.

We focused primarily on feature engineering in week 2, as we believed this was the most crucial stage for a challenge like this, and we ended up extracting 19 features (16 numerical and 3 categorical) for our train set. We trained the XGBoost model using all 19 features and obtained an MSE of **31900** on the evaluation set.

There was no specific reason behind our choice of model. Choosing a model for any challenge is the toughest task, as pinpointing a model that will yield the best result is difficult. We believe this requires a thorough and in-depth understanding of how each model works and what its key strengths are. The paper we referred to also used Gradient Boosting Machines (GBM) [19] for their work, and since the only GBM we know of is XGBoost, we chose to stick with that.

Result →

The XGBoost model trained using 19 features achieved an MSE of **29222**.

3.6.3 Week 3

We began week 3 by analysing the results and steps taken during week 2. Observing the MSE of week 1, we expected to obtain a lower MSE on the test set than the one obtained on the evaluation set. The possible drop in the MSE can be due to two reasons. Firstly, due to the incorporation of a more complex model like XGBoost. Secondly, the 18 new features were useful.

We followed the initial roadmap we made, and therefore, this week we spent most of our time performing EDA to understand different aspects of the data and which features are irrelevant to the model. This analysis can be found in section **3.5**.

After the analysis, we figured out that categorical features were not contributing as much as numerical features. Therefore, as part of feature selection, we dropped them and evaluated our model's performance. We obtained an MSE of **31600** on the evaluation set, which is a drop of around 300 from 31900.

The result aligned with our analysis. After this, we tried dropping some numerical features based on EDA and the feature importance plot from XGBoost. Despite having low correlation and importance, removing those features resulted in an increase in the MSE, which surprised us. We concluded that despite having no evidence or proof of their predictive power, those features were still contributing positively to the model's performance.

This made us realize that correlation alone doesn't reflect the predictive power of a feature. After completing feature selection, we moved on to outlier removal. The general workflow of machine learning usually involves imputing missing values first, then performing outlier removal, and finally, feature selection. However, we followed a different workflow that made more sense to us.

The workflow we followed was feature selection, followed by outlier removal, and finally imputing missing values. Our reasoning was that outlier removal could eliminate rows that have outliers in features that we would drop anyway. For example, if there are features A, B, C, and D, and we plan to drop C and D, then removing data points based on outliers in C and D would be a waste.

Additionally, imputing missing values was done after outlier removal because calculating the mean (used for imputation) could be skewed by extreme values. Therefore, removing outliers first ensures better imputation of missing values.

We considered two methods for outlier removal, which are detailed in section **3.5**. After deciding which method to use, we removed outliers and excluded the target variable in this process, since it is ethically allowed. We removed around 700 rows, leaving us with approximately 1164 rows.

With this, we concluded our work for week 3, spending most of our time on EDA. After applying outlier removal, we evaluated our model and obtained an MSE of roughly 8500. This seemed suspicious due to the drastic drop in MSE.

We hypothesized that our model was overfitting, so we checked the train and test MSE. The train MSE was around 4000, indicating potential overfitting. To address this, we used cv-split to determine the optimal `nrounds` for XGBoost, initially set to 100. The stopping condition for cv-split was set to 10.

Using cv-split, we obtained `nrounds` as 35. Evaluating the model with this new value, we surprisingly observed a further drop in the MSE instead of an increase. The test MSE was around 7400, and the train MSE was around 6500. The difference of approximately 1000 appeared reasonable, suggesting the model was not overfitting and the performance might be genuine.

Result →

The XGBoost model trained using 16 features achieved an MSE of **70239**.

3.6.4 Week 4

Just like the previous weeks, we began by starting to analyse the results and steps taken in week 3. There is a massive increase in the value of MSE. Most probably, it was outlier removal which caused this massive spike in the MSE. We first needed to decipher why this happened since outlier removal is a common step in a regression task.

At first we thought this happened because we performed outlier removal without the target variable. Since, it is ethically allowed to perform outlier removal only on features, we assumed that we removed those values from the target variable that were not outliers. In week 3, while performing outlier removal our focus was to not remove a lot of rows from the training dataset. Therefore, we let the outlier removal method remove rows based on a specific threshold.

This time we decided to refine our approach of outlier removal. Instead of removing rows which have outliers in certain features, we decided to remove rows which have outliers in the target variable. We tried this approach because predicting a very extreme delay without much exogenous data is almost impossible since the reason for those delays can only be explained by accidents or factors beyond human control. Since, our dataset had only endogenous data, we thought removing rows with extreme delays should not be a wrong move ethically.

With this we started to think the definition of an outlier in our case. The definition of delay can vary drastically depending on Geography. This is because different nations have their own definitions of a train being late or early. For instance, in a country like Japan, trains are very punctual and it is unusual to witness a delay of 10 minutes, since the average delay is around 20 seconds to 2 minutes. In fact, passengers in Japan get a train late certificate to show at their workplace that their train was late even if it was late by just 30 seconds.

On the other hand, in a country like India, a train running late by 8 hours is generally considered to be on time by the locals. So, how do we say if a delay in our training dataset is an extreme delay which can be caused only by factors beyond human control. So, we looked up for some information regarding this over the internet. According to this website [18], the average train delay is around 10 minutes in the UK. 10 minutes is equal to 600 seconds. Therefore, we removed rows where delay was more than 600 and as a result we removed 62 rows and were left with 1797 rows. We referred the distribution of

the target variable to verify this. The plot is shown in section **3.5**.

Since we did not remove a lot of rows, We decided to evaluate our model's performance again. The train MSE was around 11200 and test MSE was around 16200. As this was not a very big difference, overfitting was less likely. The performance is way better than before, however, we realised something is still wrong as we got advice an advise during our consultation that we would not witness a massive drop in the MSE.

Therefore, we analysed the min and max values of the CSV files we submitted for our predictions each week. Since we had the best score in week 2, we compared the new min and max values predicted by the model on the test set with that of the predictions we submitted during week 2. The max value in week 2 was 1871.986, in week 3 it was 204.0658 and the new max value was 471.894. The min values were more or less similar, however, the same was not the case with max values. It is clear that the new max value predicted by the model is similar to the max value of week 3 which means the performance of the model will be poor of the test set even though it is good on the evaluation set.

This gave us an indication that those extreme values are actually helping the model predict the extreme values of the test set. In fact, we are sure that most values in the test set are in same range as the train set except for some extreme values. This analysis also made us realise that why the evaluation metric for this challenge is chosen as MSE and not MAE. MSE focuses on large errors or extreme errors, which means outliers can't be ignored. We could have gotten a better MAE on the test set by dropping the outliers but since dropping them results in model not making good predictions for those observations, the MSE increases drastically.

All of these imply that we cannot perform outlier removal for this challenge as those extreme values might be outliers, but are an essential element to getting a better score on the test set. In short, the training dataset is a real representation of the test set and most likely the distribution of the values of the test set is same as the training dataset.

Since outlier removal didn't prove to be useful, we started wondering if PCA will be helpful. After some analysis we came to a conclusion that PCA will not be beneficial. This analysis has been shown in section **3.5**. At this point we felt like we have reached a dead end. There was nothing left at our disposal except for hyper-parameter tuning. We needed a break through to make some decent progress.

We then reverted back to feature engineering and thought of applying a trick one of our teammates learnt in the Machine Learning module during autumn term. The trick is to basically create polynomial or non-linear features using existing features. Since all of our features were numerical we thought this might prove to be beneficial. Therefore, we applied subtraction to create features that might represent delay in a more complex way and product to create non-linear feature. In addition, we also squared and cubed certain features to create polynomial features. With this, we created 117 new features. We refrained from creating new categorical features as our analysis during feature selection gave us an indication that categorical features will neither be useful nor contribute positively to the performance of the model.

Furthermore, we refrained from applying the division operation as after week 3 we noticed some features were having null values, and this made us wonder how that can be since the initial training dataset had no null values. Therefore, we referred to the formulas for those features and realised that the null values were caused by what we say "Division by Zero Error" in programming terms. As a result, we needed to make amendments to the formulas of those features. Alongside division, we did not use addition operation as adding 2 delays would not make any sense.

With so many features, it was obvious that there might be a lot of redundant features. Before removing redundant features, we evaluated the performance of our model with all 133 features and obtained an MSE of **31145**. This was an improvement that gave us hope that the new features might add value to the performance. By this time, we realised

that we would not be witnessing a massive drop in the MSE, and therefore, we focused on incremental results.

We were not sure whether this improvement was due to some good features or overfitting due to many redundant features. Therefore, we decided to remove the features that were highly correlated with other features. The threshold was set to 0.8 for removal. After removal, we were left with 49 features. With this we decided to apply Recursive Feature Elimination (RFE) to find the best subset of features.

The biggest issue with techniques like RFE and hyper-parameter tuning is how to effectively define the grid. If one has a really powerful system, then its a complete different story, however, if one has a system similar to us with limited resources, then stuff like RFE becomes a challenge. With so many parameters and values available, one cannot define all the values for all parameters in the grid and let the system process that forever. There must be a strategy to effectively define values in the grid. As we were not aware of any such strategy, we had to rely on random assignment of values and hope to get better results.

Unfortunately, hyper-parameter tuning did not improve the performance of our model, and therefore, we reverted back to the original settings we had before. This made us realise that hyper-parameter tuning can only guarantee improvement if the right values are defined in the grid, which will lead to improvement. Similarly, for RFE we tried to predict some numbers that might yield the best result.

Since we had 49 features, we tried multiples of 7 and some common numbers like 5, 10 and so on. RFE gave a subset of 25 features as output. The performance of the model using this new subset of 25 features turned out to be **31120**. A very minute increase of just 25 in the MSE but atleast we were sure this had fewer chances of overfitting.

Result →

The XGBoost model trained using 25 features achieved an MSE of **29816**.

3.6.5 Week 5

Like in previous weeks, we began by analyzing the results and the steps taken during Week 4. Our performance was back on track, however, it was still worse than the result obtained in Week 2. One major reason may be that we forgot to change the formula of certain features as we did for the training set, leading to null values. As a result, we had to impute those null values with the mean of the training set, causing minor deviations in our predictions.

Since our workflow seemed correct and we observed an improvement in our evaluation set, we decided to stick to our current approach. Upon reviewing the workflow, we discovered that the function to remove redundant features was not implemented correctly, and as a result, it was not working as intended. After fixing the function, we retained 61 features instead of 49.

We then performed RFE again, which returned a subset of 50 features as output. Evaluating the model with this new subset, we achieved an MSE of approximately **30,600**, an improvement of around 500. At this point, we were stuck again, trying to figure out what else could be done. We considered whether we had missed any essential steps in a typical regression workflow. However, from our perspective, we had covered everything.

With nothing else coming to mind, we decided to try an ensemble approach. Before proceeding, we wondered whether applying PCA on all 134 features would be helpful since there were many redundant features. However, PCA did not prove to be useful, as detailed in section **3.5**. With PCA ruled out, we moved on to the ensemble approach.

The next challenge was deciding which model to use. **Group C** had achieved very good results in Week 3. Upon asking, they revealed they were using GAM. While we under-

stood that merely using a new model would not guarantee better results, we had no other promising ideas, so we decided to give GAM a try. This was also a great opportunity to learn more about GAM, a model we had not used before.

We trained and evaluated the performance of GAM using the same 50 features and obtained an MSE of approximately **29,500**. With both our models ready, we proceeded to deploy the ensemble approach. The challenge here was determining the optimal weights for both models to achieve the best MSE. We plotted a graph to compare the performance of different weight combinations. Despite identifying the best combination from the plot, we chose to assign different weights based on our understanding. The plot and reasoning behind the choice of weights are explained in section **3.5**. The blended model obtained an MSE of **29000** on the evaluation set.

Result →

The blended model of XGBoost and GAM trained using 50 features achieved an MSE of **29,341**.

There is a minor decrease in the MSE which aligns with our observation on the evaluation set, however, this is still higher than the MSE we obtained in week 2 using the 19 initial features. This could mean two things. First, the original features have more impact on the performance than the derived features. Second, we have not found the best subset of features to get the best result. After 5 weeks of work on this challenge, we believe that to further decrease the MSE, much more effective feature engineering and selection needs to be done in addition to some deeper EDA which might expose certain aspects of data we have not observed until now.

In conclusion, we had more control over our steps in this challenge, since we had a basic understanding and sufficient domain knowledge. Most of our work followed the general workflow of a typical regression problem with occasional deviations.

3.7 Challenges

This small subsection aims to address some keys challenges we faced during this challenge.

1. Understanding how congestion and historical congestion are related to each other.
2. How to use the columns of the congestion variable to make features that could better capture relationships.
3. What model to use for this challenge. Since we did not have an in-depth understanding of many models, understanding which model would yield better results was a struggle.
4. How to deal with the skewness in the target variable.
5. How to create better features using the existing features or some other information in the data that could improve the performance of the model.
6. How to effectively define the grid for hyper-parameter tuning and Recursive Feature Elimination (RFE) to find the optimal subset of parameters and features.

3.8 Scope for Improvement

This small section aims to mention some things that we might have worked on or helped us improve performance.

1. We implemented a very simple blended approach, where the final prediction is basically the predictions from two models multiplied with some weight. If we had more time, we could have tried implementing a more sophisticated ensemble approach such as those we encountered during the Netflix challenge.
2. We believe that researching a bit more about the works of other people might have given us some ideas that could be implemented to improve performance.
3. Performing some EDA on the congestion of specific hours might have given us new ideas of encoding the hour feature. We got this idea after seeing the presentation of **Group E**.

3.9 Conclusion

1. More exogenous data is needed to make better predictions.
2. Delays at stations closer to Nottingham had a stronger impact on the final delay compared to those further away.
3. Effective feature engineering is crucial for a challenge like this.
4. Numerical features had a more significant impact on the performance than the categorical features.
5. Removing outliers initially reduced error on the training set, however, it negatively affected performance on the test set, indicating that sometimes outliers are not outliers and can be a real representation of the test set.
6. Although PCA captured 88% variance with 20 components, it did not lead to performance improvement, suggesting that the original features contained diverse and meaningful information.
7. PCA does not guarantee better results just because there is a lot of variance in the data or the features are highly correlated.
8. High correlation between features does not mean that the features contain redundant information.
9. An additive model like GAM outperformed a tree-based model like XGBoost on the evaluation set, despite the fact that tree-based models are considered better when it comes to handling skewness in the data. This shows that having an in-depth understanding about the working of the models can prove to be beneficial since it helps to choose the best model for any challenge.
10. A blend of XGBoost and GAM with manual weights provided better results on the evaluation set, improving performance over the standalone models.